SpringBoot 面试题速记通关版 _ 面试刷题 mianshiya.com

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*

Spring Boot 工程启动以后，我希望将数据库中已有的固定内容，打入到 Redis 缓存中，请问如何处理？

应用启动后加载数据到 Redis，关键是找到合适的**钩子时机**，既不能太早（比如数据库连接还没建好），也不能太晚（业务请求已经开始进来了）。

Spring 提供了 `CommandLineRunner` 和 `ApplicationRunner`，这两个接口的实现会在 Spring Boot 容器初始化完成后自动执行。用它们来触发缓存预热是最稳妥的选择。

1）定义一个组件实现 `CommandLineRunner`，在 `run` 方法里调用你的缓存预热逻辑

```
@Component
public class CachePreloader implements CommandLineRunner {
    private final UserService userService;
    private final StringRedisTemplate redisTemplate;

    public CachePreloader(UserService userService, StringRedisTemplate
redisTemplate) {
        this.userService = userService;
        this.redisTemplate = redisTemplate;
    }

    @Override
    public void run(String... args) {
        List<User> users = userService.getAllUsers();
        users.forEach(user ->
            redisTemplate.opsForValue().set("user:" + user.getId(),
JSON.toJSONString(user))
        );
    }
}
```

2）如果数据量大，建议分页加载，避免一次性查 5 万条压垮数据库。可以按每页 1000 条异步刷入。

3）注意异常处理。如果 Redis 挂了，要不要阻塞启动？一般来说允许启动继续，但要打告警日志，后续通过定时任务补偿。

4）更复杂的场景可以用 `@EventListener(ContextRefreshedEvent.class)` ，它比 `CommandLineRunner` 更底层一点，适合多模块协同触发。

整个过程其实就是利用 Spring 的生命周期机制，在**容器就绪后、服务对外前**把脏活累活干完。

Spring Boot 配置文件加载优先级你知道吗？

Spring Boot 启动时会按固定顺序扫描多位置的配置文件，后加载的会覆盖前面的同名属性。整个过程不难理解，关键是记住几个核心路径和优先级次序。

外部配置优先级最高，比如通过命令行参数指定的配置，它能直接覆盖所有其他来源。其次是当前项目目录下的 `config` 子目录里的 `application.properties` 或 `application.yml` ，这是给运维留的动态调整入口。

接着是项目根目录本身，这里的配置文件虽然也能被识别，但通常不如放在 `config` 目录下规范。再往下就是 jar 包内部的 `classpath:/config/` 和 `classpath:/` ，也就是我们开发时最常见的 `src/main/resources` 下的配置。

1）命令行参数（--server.port=8081）
2）项目根目录的 `config/` 目录
3）项目根目录
4）classpath 的 `config/` 目录
5）classpath 根路径

如果同时存在 yml 和 properties 文件，**同目录下** properties 会覆盖 yml。另外，profile 特定文件如 `application-prod.yml` 只在对应环境激活时才生效，它的优先级高于通用 `application.yml` 。

实际部署时，推荐把默认配置打在 jar 里，运维用外部 config 目录或启动参数调优，这样既安全又灵活。

Spring Boot 是否可以使用 XML 配置？

Spring Boot 虽然主打注解和自动配置，但完全支持 XML 配置，只是不推荐作为首选。

1）你可以用 `@ImportResource` 注解导入传统的 Spring XML 文件。比如你的老项目有一堆 `applicationContext.xml` 定义的 Bean，直接在启动类上加一句 `@ImportResource("classpath:applicationContext.xml")` 就能加载进去。

2）XML 在一些复杂场景下仍有价值，比如整合老系统、需要精确控制 Bean 的生命周期或使用某些只提供 XML 配置的第三方库时。像一些银行的老系统迁移，可能还在用 IBM MQ 的 XML 配置方式。

3）但日常开发中，你应该优先用 Java Config 和 `application.yml` 。XML 写起来冗长，维护成本高，而且破坏了 Spring Boot 约定优于配置的原则。新功能基本都通过注解扩展，比如 `@ConditionalOnMissingBean` 这种条件装配，XML 压根搞不定。

代码示例：

```
@SpringBootApplication
@ImportResource("classpath:beans.xml")
public class App {
    public static void main(String[] args) {
```

```
        SpringApplication.run(App.class, args);
    }
}
```

总的来说，XML 是后备选项，不是主流玩法。你能用它，但别滥用。

SpringBoot 中如何实现定时任务？

SpringBoot 里做定时任务，最常用的就是 `@Scheduled` 注解，配合 `@EnableScheduling` 启用。整个过程其实挺轻量的，框架帮你把线程池这些脏活累活都包了。

1）先在启动类或配置类上加 `@EnableScheduling`，开启定时功能。

2）在方法上用 `@Scheduled` 标记，指定执行策略。支持固定频率、固定延迟，还有 cron 表达式。

比如每5秒跑一次：

```
@Scheduled(fixedRate = 5000)
public void task() {
    System.out.println("执行中...");
}
```

cron 更灵活，像每天凌晨1点：

```
@Scheduled(cron = "0 0 1 * * ?")
public void dailyJob() { ... }
```

默认是单线程串行执行，所有任务共用一个线程池。如果某个任务耗时长，后面的会阻塞。想并行就得自己定义 `TaskScheduler` 或 `ThreadPoolTaskScheduler`，给多几个线程。

集群环境下 **@Scheduled** 就搞不定了，多个实例会重复执行。这时候得引入分布式调度方案，比如 XXL-JOB、Elastic-Job 或 Quartz 配合数据库锁。它们能保证同一时间只有一个节点执行任务。

简单应用直接用注解就行，开发快又够用。但一旦上集群，就得靠外部调度中间件来协调，不然任务重复执行可能引发数据问题。

@Async 什么时候会失效？

方法调用在同一个类中会导致代理失效，这是最常见的情况。Spring 的 @Async 基于 AOP 代理实现，只有外部 bean 调用方法时才会走代理逻辑，进而触发异步执行。

如果本类方法直接调用被 @Async 标记的方法，相当于 this.method()，压根不经过代理对象，异步机制就失效了。

1）确保调用方和被调用方是不同的 Spring Bean
2）@Async 方法不能是 private、static 或 final 的，代理无法覆盖这些方法

3）所在类必须被 Spring 容器管理，普通 new 出来的对象无效
4）需要启用 @EnableAsync，否则注解不生效
5）异常未处理可能导致线程池任务中断，表面看像是“没执行”

代码示例：

```
@Service
public class AsyncTaskService {
    @Async
    public void sendEmail() {
        System.out.println("当前线程: " + Thread.currentThread().getName());
    }
}

@Service
public class BusinessService {
    @Autowired
    private AsyncTaskService taskService;

    public void process() {
        taskService.sendEmail(); // 正确：通过代理调用
    }
}
```

错误用法就是把 taskService 的调用换成 this.sendEmail()。

说说 Springboot 的启动流程？

Spring Boot 启动看起来就一行 `run()` 调用，背后其实做了不少事。整个过程从创建上下文到刷新容器，每一步都封装好了，但得知道它干了啥。

1）启动时先初始化一个 `SpringApplication` 实例，这里会做些前置判断，比如当前项目类型是 Servlet 还是 Reactive，然后设置默认的 `ApplicationContextInitializer` 和 `ApplicationListener` 。

2）真正 `run()` 的时候，第一步是准备环境，不管是 Web 环境还是非 Web 环境，都会读取配置文件（application.yml 或 properties），也支持命令行参数覆盖。

3）接着创建合适的应用上下文 `ApplicationContext` 。如果是 Web 项目，就是 `AnnotationConfigServletWebServerApplicationContext` ，这个上下文能启动内嵌容器。

4）上下文准备好后，开始加载 Bean。Spring Boot 会扫描主类所在包下的所有组件，靠的是 `@SpringBootApplication` 里的 `@ComponentScan` 。

5）最关键的一步是**刷新上下文**，这一步触发内嵌服务器启动。比如 Tomcat 就是在 `onRefresh()` 时被拉起来的，端口绑定、连接器配置都在这阶段完成。

6）最后发布 `ApplicationReadyEvent` ，表示服务已经可以接收请求了。

代码上其实就是：

```
public static void main(String[] args) {
    SpringApplication.run(Application.class, args);
```

```
}
```

但你知道它背后把自动配置、条件装配、内嵌容器这些脏活累活全包了，才让你能一键启动。

SpringBoot 是如何实现自动配置的？

SpringBoot 的自动配置不是魔法，靠的是约定大于配置的思想和几个关键机制组合起来的。

启动时会扫描 classpath 下所有 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` 文件，里面列了一堆 **自动配置类**。这些类不会无脑加载，每个都带着条件注解，比如 `@ConditionalOnClass` 、`@ConditionalOnMissingBean` ，只有满足条件才生效。

举个例子，你引入了 `spring-boot-starter-data-redis` ，classpath 里有了 `LettuceConnectionFactory` ，自动配置发现这个类存在，而且容器里还没人创建过 RedisTemplate，它就帮你把连接池、模板对象都配好。

1） 自动配置类通常放在 `spring.factories` 或 `org.springframework.boot.autoconfigure.AutoConfiguration.imports` 中 2） 条件装配确保不冲突，比如只在某个类存在且某个 Bean 缺失时才创建 3） 配置优先级可通过 `@AutoConfigureAfter` 控制

别小看 `@EnableAutoConfiguration` 这个注解，它就是触发整个链路的开关。实际项目里像 Nacos、MyBatis-Plus 的 starter 都是照这个套路来，把一堆初始化逻辑封装掉，你写个依赖就能用。

SpringBoot（Spring）中为什么不推荐使用 @Autowired？

Spring 官方其实在新版本里把 `@Autowired` 标记为可选了，根本原因是指定注入方式时太死板，容易引发隐性问题。

1） 依赖注入最稳的方式是构造器注入。用构造函数把依赖写清楚，对象一创建就必须把该给的都给全，这样单元测试也好写，类的状态也明确。你用 `@Autowired` 注解在字段上，看似省事，但实际掩盖了依赖关系，别人看代码不知道这玩意从哪来。

2） 字段注入会导致 `final` 关键字没法用，类的不可变性被破坏。而构造器注入天然支持 `final` ，更安全。

3） 循环依赖场景下， `@Autowired` 会让人踩坑。虽然 Spring 能靠三级缓存搞一下单例的循环依赖，但这其实是种坏味道，暴露的是设计问题。过度依赖 `@Autowired` 会让这种问题更难发现。

4） 在非单例场景，比如 prototype bean 注入到 singleton 里，字段注入可能拿到过期实例，而方法注入（lookup method）或 `ObjectProvider` 才是正解，但 `@Autowired` 让人忽略这些细节。

现在推荐的做法是在构造函数上不加注解，靠 IDE 提示自动装配。Spring Boot 2.6+ 默认关闭循环依赖后，构造器注入成了事实标准。

说白了，`@Autowired` 把脏活累活藏起来了，但架构设计上得往前看一步。

如何在 SpringBoot 启动时执行特定代码？有哪些方式？

SpringBoot 启动时执行代码，本质上是利用它的生命周期回调机制。你不需要手动触发，框架会在启动的不同阶段自动调用你注册的逻辑。

最常见的办法是实现 `CommandLineRunner` 接口。只要你的类加上 `@Component`，重写 `run` 方法，它就会在容器初始化完成后执行。参数就是应用启动时传入的命令行参数。

```
@Component
public class MyStartupTask implements CommandLineRunner {
    public void run(String... args) {
        System.out.println("启动任务执行");
    }
}
```

如果多个 Runner 有先后依赖，可以用 `@Order(1)` 或让类实现 `Ordered` 接口来控制顺序。

另一种方式是监听 `ApplicationReadyEvent` 事件。这代表整个应用已经准备就绪，包括所有 Bean 都加载完了，数据库连接也建好了。适合做最后的初始化通知或健康检查上报。

```
@EventListener(ApplicationReadyEvent.class)
public void onAppReady() {
    System.out.println("应用已就绪");
}
```

如果你需要更底层的控制，比如想在刷新容器早期就介入，可以考虑 `ApplicationRunner`，它和 `CommandLineRunner` 类似，但接收的是封装后的 `ApplicationArguments`，处理参数更方便。

还有一种情况是你要改配置或环境变量，就得用 `ApplicationContextInitializer`，它在上下文创建后、刷新前执行，一般配合 `spring.factories` 文件注册。

总的来说，90% 的场景用 `CommandLineRunner` 就够了。真要分阶段，再考虑事件监听或者 initializer。别把简单事搞复杂，脏活累活框架都帮你扛住了。

Spring Boot 中如何实现异步处理？

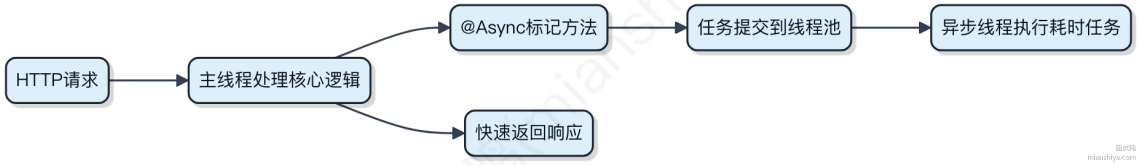
Spring Boot 的异步处理主要靠 @Async 注解和线程池配合，把耗时操作从主线程剥离，避免阻塞请求响应。

1）先在启动类或配置类上加 @EnableAsync 开启异步支持。然后在需要异步执行的方法上标注 @Async，这个方法就会被动态代理拦截，交给独立线程去跑。

2）默认用的是 Spring 内置的 SimpleAsyncTaskExecutor，但生产环境一定得自定义线程池。比如用 ThreadPoolTaskExecutor 配置核心线程数、队列容量这些参数，不然容易把系统搞崩。常见的像发短信、写日志这种非关键路径操作，就适合扔进异步处理。

3）返回值可以是 void 或 Future / CompletableFuture。如果要获取结果，推荐用 CompletableFuture，它支持链式调用和组合，跟业务逻辑融合更顺。

举个例子，用户注册后异步发送欢迎邮件，主线程完成注册直接返回，邮件任务由专门的线程池处理，这样接口响应能从几百毫秒降到几十毫秒。



注意事务和异常的处理。@Async 方法别直接抛异常，最好内部 try-catch 记录日志，因为调用方可能已经走完了。另外，同一个类里方法直接调用不会触发代理，得拆到不同 Bean 里。

在 Spring Boot 中如何实现多数据源配置？

配置多数据源主要解决一个应用需要访问多个数据库的场景，比如订单库和用户库分离。关键在于让 Spring 知道哪个方法该用哪个数据源。

Spring Boot 默认只配一个数据源，多个就得手动干预。核心是**动态数据源路由**，用 `AbstractRoutingDataSource` 做分发，根据当前上下文决定用哪个数据源。

一般做法是定义一个 `@DataSource` 注解，标记在 service 方法上，比如 `@DataSource("slave1")` 。然后通过 AOP 拦截这些方法，把数据源标识存到 `ThreadLocal` 里。动态数据源在查询时从 `ThreadLocal` 拿标识，返回对应的数据源实例。

配置上，先定义两个 `DataSource` Bean，比如 `masterDataSource` 和 `slaveDataSource` ，注册到一个 `DynamicDataSource` 里，再把这个 `DynamicDataSource` 放到 Spring 容器作为主数据源。事务管理器也得指定用这个动态数据源。

注意事务要小心，如果用了 `@Transactional` ，默认绑定的是主数据源，跨数据源事务搞不定，只能保证单个库的 ACID。

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface DataSource {
    String value();
}

// AOP 切面设置数据源
@Before("@annotation(ds)")
public void setDataSource(DataSource ds) {
    DynamicDataSourceContextHolder.set(ds.value());
}
```

整个流程就是：注解打标 → AOP 拦截设上下文 → 动态数据源读上下文选库。

说说你对 Spring Boot 事件机制的了解？

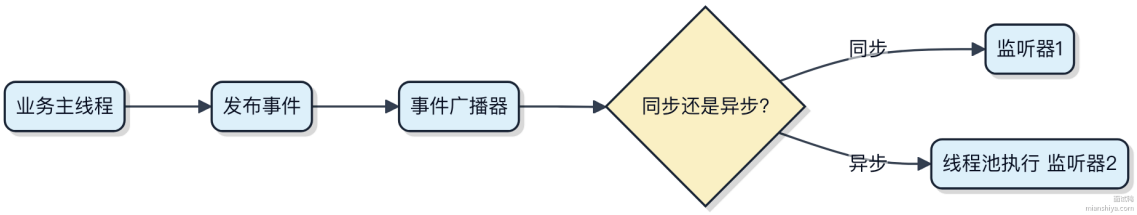
Spring Boot 事件机制本质是观察者模式的落地，用来解耦业务逻辑。比如用户注册成功后要发邮件、加积分，这些后续动作就可以用事件来触发，主流程不用硬编码调用。

应用启动时，Spring 容器会自动识别 `ApplicationListener` 或标注 `@EventListener` 的方法，把它们注册成监听器。你发一个事件，比如 `context.publishEvent(new UserRegisteredEvent(user))` ，容器就会找到所有监听这个类型的处理器去执行。

监听方法默认是同步阻塞的，主事务没提交前其他监听器拿不到数据。如果想异步处理，得在类上加 `@EnableAsync` ，然后给监听方法加上 `@Async` ，这时候要注意事务边界问题，别在事务还没提交时就消费了未提交的数据。

事件类型可以自定义，只要继承 `ApplicationEvent` 就行，但其实从 Spring 4.2 开始，连这一步都省了，直接发任意对象当事件也行。

1）事件发布和监听都在同一个应用上下文内，不跨 JVM
2）适合单体内部模块解耦，跨服务还得靠消息队列比如 RabbitMQ、Kafka
3）别滥用，频繁发事件会影响性能，毕竟每个监听都是反射调用



常见坑是异步监听导致事务不一致，或者监听器抛异常让主流程跟着挂。可以包装一层 try-catch，或者用 `@EventListener` 的 `errorHandler` 属性指定错误处理器。

Spring Boot 2.x 与 1.x 版本有哪些主要的改进和区别？

Spring Boot 2.x 相比 1.x 不是小修小补，是一次全面升级，底层支撑和开发体验都变了。

JDK 版本要求直接跳到 8+，不再支持 6 和 7。这意味着你可以放心用 Lambda、Stream 这些特性，不用再写一堆匿名内部类。内嵌容器默认版本也升了，Tomcat 8.5+ 成标配，Servlet 规范跟着到了 3.1+。

自动配置机制更智能了。条件化配置的判断更细，比如 `@ConditionalOnMissingBean` 现在能按类型和名称双重判断，避免误覆盖。配置元数据支持 IDE 提示， `application.yml` 里打 `spring.` 就能自动联想，写配置不再靠背。

WebFlux 加入让响应式编程成为一等公民。你可以用 `@Controller` + `Mono/Flux` 写非阻塞接口，配合 Netty 或 Servlet 3.1+ 容器跑。这在高并发 IO 场景下能显著降低线程开销，像网关类服务用 WebFlux 能扛住更多连接。

Actuator 变得更安全也更实用。端点默认只暴露 `/health` 和 `/info` ，其他要手动开启。HTTP 端点路径从 `/metrics` 这种变成 `/actuator/metrics` ，隔离更清晰。还支持 Prometheus 格式输出，对接 Grafana 面板零成本。

依赖管理上，默认用 Spring Framework 5，Reactor 成核心响应式库。如果你用 WebClient 替代 RestTemplate，天然支持异步非阻塞调用，和 Spring Cloud Gateway 配合更顺。

```
@Bean
public RouterFunction<ServerResponse> route(UserHandler handler) {
    return route(GET("/users/{id}"), handler::getById);
}
```

这套函数式路由写法，是 WebFlux 提供的另一种选择，适合轻量接口场景。

什么是 Spring Actuator？它有什么优势？

Spring Actuator 是 Spring Boot 项目里的一个模块，它给应用加了一层监控和管理能力。你不用改业务代码，只要引入 starter，就能通过 HTTP 或 JMX 暴露一些预定义的端点（endpoints），比如查看健康状态、内存使用、线程堆栈、请求指标等。

这些端点能帮你快速定位生产问题。比如服务挂了，调用 `/actuator/health` 一看就知道是数据库连不上还是 Redis 超时。再比如流量突增，用 `/actuator/metrics/jvm.memory.used` 结合 Grafana 能看出内存是不是在缓慢上涨。

1）暴露的端点默认有安全限制，生产上必须配合认证开启，避免信息泄露
2）可以自定义端点，比如加个 `/actuator/feature-toggle` 来动态开关功能
3）和 Micrometer 集成后，能把指标推送到 Prometheus，实现全链路监控

代码上就两步：加依赖，开端点。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
```

其实它的价值不在功能多炫，而是统一了微服务的观测入口。你在用 Spring Cloud Gateway 还是 Eureka，所有服务的健康检查、指标格式都是一致的，运维脚本和告警规则能复用，省掉一堆脏活累活。

什么是 Spring Boot？

Spring Boot 本质是 Spring 框架的脚手架，目的就是把开发从繁琐的配置里解放出来。你不用再手动写一堆 XML 去配数据源、MVC、事务，它通过**自动装配**机制帮你搞定。

启动一个 Web 项目，传统 Spring 要配 `DispatcherServlet` 、组件扫描、视图解析器，可能得上百行配置。Spring Boot 只要加个 `@SpringBootApplication` ，main 方法跑 `SpringApplication.run()` 就行了。比如用 `spring-boot-starter-web` ，一引入，Tomcat 和 Spring MVC 自动就位。

它采用“约定大于配置”的思路。比如默认静态资源放在 `/static` ，内嵌 Tomcat 监听 8080 端口，这些都不用额外说明。如果想改，再通过 `application.properties` 或 `application.yml` 覆盖就行。

依赖管理也是省心的地方。你用 `spring-boot-starter-parent` 作为父 POM，里面已经定好了常用库的版本号，避免自己引入时版本冲突。像整合 MyBatis、Redis、Kafka，基本都是引入对应的 starter，写点配置，直接开干。

这种快速搭建能力，让微服务架构落地变得简单。比如用 Spring Cloud + Spring Boot，几十个服务也能快速统一技术栈。不过也得注意，自动装配虽然方便，但出了问题如果不熟悉底层原理，调试起来会有点懵。

所以它不是替代 Spring，而是让 Spring 更容易上手和维护。小到单体应用，大到云原生服务，都能扛住。

在 Spring Boot 中你是怎么使用拦截器的？

拦截器在 Spring Boot 里其实就是一个能插在请求处理流程中的钩子，你可以在 Controller 执行前、执行后、视图渲染前这些节点干点事，比如鉴权、日志、性能统计。

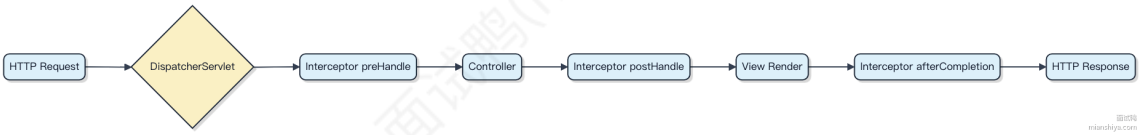
要加拦截器，得先写个类实现 HandlerInterceptor 接口，把 preHandle、postHandle、afterCompletion 这几个方法按需重写。preHandle 最常用，返回 false 就会直接中断请求。

接着你需要注册它，新建一个配置类实现 WebMvcConfigurer，然后重写 addInterceptors 方法。这里你可以把自定义的拦截器加进去，还能用 ant 匹配路径，比如只对 `/api/**` 生效，排除静态资源。

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new AuthInterceptor())
                .addPathPatterns("/api/**")
                .excludePathPatterns("/login", "/static/**");
    }
}
```

注意一点，拦截器是跟着 WebMvc 走的，所以它**压根不经过** Filter 那层，也不影响 Servlet 的生命周期。如果你需要处理跨域、字符编码这种底层问题，还是得用 Filter。

1）拦截器适合做业务无关但通用的逻辑，像登录检查、接口耗时打印
2）多个拦截器有顺序，注册时的顺序就是执行顺序
3）异常发生时 postHandle 不会触发，善用 afterCompletion 做资源清理



Spring Boot 如何处理跨域请求（CORS）？

Spring Boot 处理跨域请求主要靠 **CORS** 协议支持，浏览器在发请求前会先发一个 OPTIONS 预检请求，后端得正确响应才能继续。

1）最简单的方式是用 `@CrossOrigin` 注解，加在 Controller 或方法上，指定允许的源、方法和头。比如：

```
@RestController
@CrossOrigin(origins = "http://localhost:3000")
```

```
public class ApiController {
    @GetMapping("/data")
    public String getData() {
        return "hello";
    }
}
```

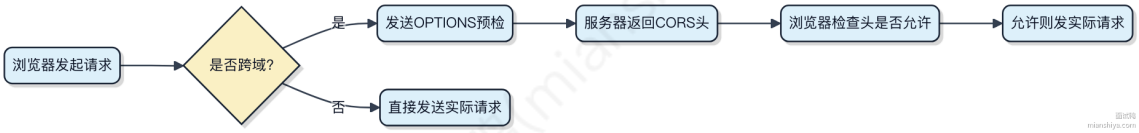
2）全局配置推荐实现 `WebMvcConfigurer` 的 `addCorsMappings` 方法，统一管理规则：

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins("http://localhost:3000")
                .allowedMethods("GET", "POST", "PUT", "DELETE")
                .allowCredentials(true)
                .maxAge(3600);
    }
}
```

3）如果项目用了 Spring Security，记得确保安全链放行 OPTIONS 请求，否则预检会被拦截。可以在 `HttpSecurity` 里配置：

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
    // 其他规则
);
```

实际项目中，一般前端域名固定，用全局配置就够了。如果是开放 API，可能还要动态匹配 origin，这时候可以写个过滤器手动设置 header，比如 `Access-Control-Allow-Origin` 。



如何理解 Spring Boot 中的 starter?

Spring Boot 的 starter 本质是一组依赖描述符，帮你快速集成某个功能模块，不用自己一个个找依赖版本。

比如你要用 Redis，直接引入 `spring-boot-starter-data-redis` ，它背后已经帮你组合好了 Lettuce 或 Jedis、Spring Data Redis、连接池、序列化等依赖，版本也都对齐了。你压根不需要关心具体引入了哪些库，也不用操心版本冲突。

1）所有 starter 都遵循统一命名规范： `spring-boot-starter-xxx` ，第三方框架接入也按这规则，比如 `mybatis-spring-boot-starter` 。

2）starter 里不写业务逻辑，主要是 `pom.xml` 的依赖管理。真正的自动配置由对应的 autoconfigure 模块实现，比如 `spring-boot-autoconfigure` 里的 `RedisAutoConfiguration` 会根据类路径上的组件自动创建

`RedisTemplate` 、 `LettuceConnectionFactory` 。

3）自动装配基于条件注解，像 `@ConditionalOnClass` 、 `@ConditionalOnMissingBean` ，只有满足条件才生效。比如发现 classpath 有 `Jedis.class` 才去配置 Jedis 连接工厂。

4）你可以自定义 starter，拆成两部分： `xxx-spring-boot-starter` （空壳，引 autoconfigure），和 `xxx-spring-boot-autoconfigure` （核心配置，含 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` 文件声明配置类）。

常见的像 `spring-boot-starter-web` 带了 Tomcat、Spring MVC、Jackson； `spring-boot-starter-jdbc` 自动配好 DataSource 和 JdbcTemplate。省去了大量基础配置的脏活累活，项目一启动就能用。

SpringBoot 默认同时可以处理的最大连接数是多少？

SpringBoot 内嵌的 Tomcat 默认最大线程数是 200，也就是说它默认能同时处理 200 个请求。每个请求对应一个线程，这个值由 `server.tomcat.max-threads` 控制。

1）当并发请求数超过 200，新的请求会进入等待队列，队列长度由 `server.tomcat.accept-count` 决定，默认是 100。超过这个总数，客户端就会收到连接被拒绝的响应。

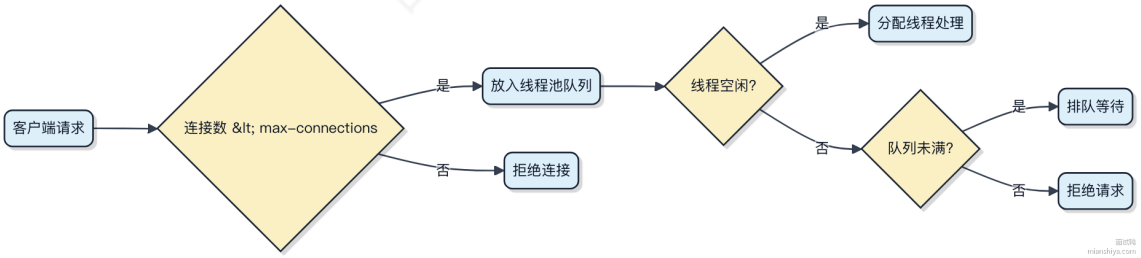
2）连接建立的最大数量还受限于 `server.tomcat.max-connections` ，默认值是 8192。这是指服务器最多能保持多少个 TCP 连接，包括正在处理的和在队列里等着的。

3）这些参数可以根据实际场景调整。比如高并发场景可能需要把 max-threads 调到 500 或更高，同时注意线程多了会增加上下文切换开销。

代码上不需要特别配置，直接在 application.yml 里改：

```yaml
server:
  tomcat:
    max-threads: 500
    accept-count: 200
    max-connections: 10000
```

换个角度看，如果用的是 Undertow 或 Netty，那对应的参数名和默认值就不同了。比如 Undertow 的默认工作线程是 8 + CPU 核数，IO 线程是 2 倍 CPU 核数。



Spring Boot 打成的 jar 和普通的 jar 有什么区别？

Spring Boot 打出来的 jar 和普通 jar 最大的不同在于**可执行性**和**依赖打包方式**。

普通 jar 只是把 .class 文件按包结构打进去，运行得靠外部 classpath 引入依赖。你用 `java -jar` 跑一个普通 Spring 项目 jar，基本会报错找不到主类或者依赖。

Spring Boot 的 jar 是个**胖 jar（fat jar）**，它把项目所有的编译类 + 依赖的第三方库 + 内嵌容器（比如 Tomcat）全塞进一个 jar 里了。启动的时候，通过自定义的类加载器 `LaunchedURLClassLoader` 动态加载内部的 jar 包。

它的目录结构也特别，除了正常的 `/classes` ，还有 `/lib` 目录存着所有依赖 jar， `/BOOT-INF/classes` 放项目自己的 class， `/META-INF/MANIFEST.MF` 里的 Main-Class 指向的是 `org.springframework.boot.loader.JarLauncher` ，不是你写的 Application 类。

这个 JarLauncher 就是启动入口，它会先解析内部结构，把 `/BOOT-INF/lib` 下的 jar 全加到类路径，再反射调用你配置的真正启动类。

代码上不用你操心，只要在 pom.xml 用了 `spring-boot-maven-plugin` ，build 时就会自动处理。

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

所以一句话：普通 jar 是模块包，Spring Boot jar 是**自带容器、开箱即用的部署包**，适合微服务这种独立部署场景。像 Nacos、Eureka 的客户端启动包也是这么打的。

如何在 Spring Boot 中定义和读取自定义配置？

Spring Boot 读自定义配置，本质是把配置文件里的值映射到 Java 对象里。最常用的方式就是 `@ConfigurationProperties` ，它能把配置自动绑定到类的字段上。

比如你在 `application.yml` 里加了：

```
app:
  name: user-service
  timeout: 5000
```

对应写个类：

```
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;
    private long timeout;
```

```
    // getter/setter
}
```

只要加上 `@Component` ，这个类就能在其他地方 `@Autowired` 注入使用。字段名会自动和配置项匹配，支持松散绑定（比如 `timeoutMs` 可以对应 `timeout-ms` ）。

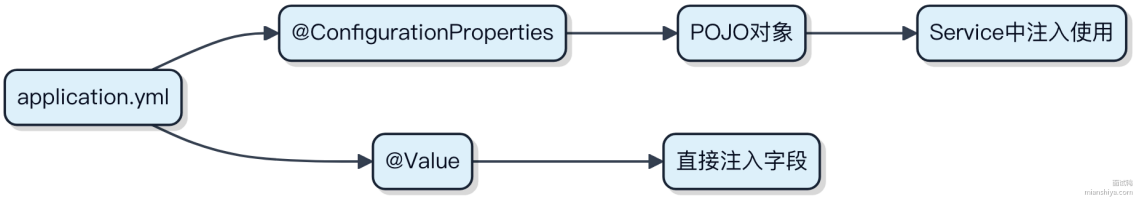
如果不想用 `@Component` ，也可以在配置类里用 `@Bean` 手动注册，然后照样能用 `@ConfigurationProperties` 绑定。

还有一种方式是 `@Value("${xxx}")` ，适合读单个值：

```
@Value("${app.timeout}")
private long timeout;
```

但 `@Value` 不支持复杂对象、不支持松散绑定、也不支持 JSR-303 校验，灵活性差很多。真要搞配置对象，还是得靠 `@ConfigurationProperties` 。

它支持校验，比如加上 `@Validated` 就能用 `@NotNull` 这些注解。也支持嵌套结构，List/Map 类型也能处理。



Spring Boot 中 application.properties 和 application.yml 的区别是什么？

Spring Boot 配置文件选 `properties` 还是 `yml` ，主要看结构和可读性。

1）格式上， `application.properties` 是键值对形式，层级深了就得重复写前缀。比如：

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/test
spring.datasource.username=root
spring.datasource.password=123456
```

2）而 `application.yml` 用缩进表示层级，结构更清晰，尤其适合复杂配置：

```
server:
  port: 8080
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/test
    username: root
    password: 123456
```

3）功能上两者完全等价，Spring Boot 都支持，也可以共存。但注意同名配置项会冲突，**加载顺序**决定了优先级：`properties` 会覆盖 `yml` 中的相同配置。

4）实际项目里，团队偏向简洁就用 `yml` ，毕竟像 `Spring Cloud` 、 `Kubernetes` 这些生态都习惯用 YAML。但如果对缩进敏感，怕空格搞错导致解析失败，那 `properties` 更稳妥。

别忘了，除了这两个，还能通过环境变量、命令行参数等方式注入配置，灵活性很高。

Spring Boot 支持哪些嵌入 Web 容器？

Spring Boot 开箱就支持好几种嵌入式 Web 容器，最常用的其实是三个：Tomcat、Jetty 和 Undertow。

1）默认是 **Tomcat**，你只要引入 `spring-boot-starter-web` ，它自带了 Tomcat 作为嵌入容器，启动后就能直接对外服务，不用额外部署 WAR 包。

2）如果你想用 Jetty，比如在高并发长连接场景下压测表现更好，那就把 starter 里的 Tomcat 依赖排除，加上 `spring-boot-starter-jetty` 。像 Spring Cloud Gateway 有些项目就会切到 Jetty 或 Undertow。

3）Undertow 是 Red Hat 出的，性能不错，内存占用也低，适合想压榨服务器资源的场景。换起来也简单，排除 Tomcat 后引入 `spring-boot-starter-undertow` 就行。

它们都是通过 `ServletWebServerFactory` 接口来创建具体容器实例的，Spring Boot 自动配置会根据 classpath 上的类选择合适的实现。比如看到 `org.apache.catalina.startup.Tomcat` 就上 Tomcat，看到 `io.undertow.Undertow` 就初始化 Undertow。

代码上你完全不用改，切换靠依赖管理。举个 Maven 的片段：

```
<exclusions>
    <exclusion>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
</exclusions>
```

加上对应的 starter 就能替换。一般小项目用默认 Tomcat 完全够用，搞微服务网关或者追求极致性能再考虑换。

Spring Boot 是如何通过 main 方法启动 web 项目的？

Spring Boot 启动 Web 项目不是靠 main 方法本身多高级，而是靠 `SpringApplication.run()` 这个静态方法把一堆脏活累活给干了。

main 方法里通常就两行代码，核心是调用 `SpringApplication.run(Application.class, args)` 。它会做几件事：推断应用类型，比如有没引入 Tomcat、Jetty 这些依赖；加载所有 `META-INF/spring.factories` 里配置的初始化器和监听器；最后创建对应的上下文容器。

如果是 Web 应用，它会创建 **AnnotationConfigServletWebServerApplicationContext**，这个上下文知道怎么启动内嵌 Web 服务器。在刷新阶段，会触发 `ServletWebServerFactoryAutoConfiguration` 自动装配，根据 classpath 选一个可用的 Web 服务器，比如默认是 Tomcat。

1）先找有没有 `ServletWebServerFactory` 类型的 Bean
2）通过工厂获取 `WebServer` 实例，比如 `TomcatServletWebServerFactory`
3）调用 `webServer.start()` 把内嵌容器跑起来

```
@SpringBootApplication
public class App {
```

```
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

整个过程最关键是自动装配 + 内嵌容器 + 特殊上下文联动。你不需要手动 new Tomcat 或 configure connector，全靠条件化配置搞定。只要加了 `spring-boot-starter-web` ，启动时就会拉起一个预配置好的 Web 服务，监听 8080。

Spring Boot 的核心特性有哪些？

Spring Boot 的核心特性其实围绕一个目标：让 Spring 应用能快速落地，不用在配置上折腾半天。

1）**自动装配**是它最硬的底气。你加个 `spring-boot-starter-web` ，它就帮你把 Tomcat、Spring MVC 都配好了，不需要写一堆 XML 或 `@Configuration` 。背后的逻辑是 `@ConditionalOnClass` 这些条件注解，只要类路径有某个类，就自动生效对应配置。

2）起步依赖（Starter）设计得很聪明。比如用 Kafka 就引入 `spring-kafka-starter` ，它把常用的依赖版本都锁定了，不会出现版本冲突。常见的像 `spring-boot-starter-data-jpa` 、 `spring-boot-starter-actuator` 都是这种模式。

3）内嵌容器让部署变简单。以前要打 WAR 包扔到外部 Tomcat，现在直接 `java -jar` 启动，因为内嵌了 Tomcat、Jetty 或 Undertow。请求进来时，Servlet 容器直接加载应用，压根不经过外部部署。

4）Actuator 提供生产级监控。比如 `/actuator/health` 看健康状态， `/actuator/metrics` 查 JVM 指标，结合 Prometheus 和 Grafana 能搞监控大盘。

5）外部化配置支持多环境。 `application.yml` 里写 `spring.profiles.active=prod` ，就能切换不同配置，本地和线上互不影响。

代码示例就是最简单的启动类：

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

这个注解本身是个组合注解，包含了 `@Configuration` 、 `@EnableAutoConfiguration` 和 `@ComponentScan` 。

Spring Boot 3.x 与 2.x 版本有哪些主要的改进和区别？

Spring Boot 3 是一次真正意义上的大版本跃进，不是简单功能叠加。最关键是它全面拥抱了 Java 17+ 和 Jakarta EE 9+，这意味着包路径从 `javax.*` 全面迁移到了 `jakarta.*` 。你如果用老的 `javax.servlet` 相关依赖，直接启动就报错，必须升级配套中间件。

Java 版本要求从 8 跳到 **17**，官方推荐使用 LTS 版本开发。这带来了很多语言层面的新特性可用，比如 record、sealed class、模式匹配等，框架内部也大量使用这些特性简化代码。

GraalVM 原生镜像支持是另一个重头戏。Spring Boot 3 配合 Spring Native 可以把应用编译成原生可执行文件，启动时间从秒级降到毫秒级，内存占用减少一半以上。像云函数、Serverless 这种场景特别适合。

1）自动配置机制更智能，条件注解支持延迟评估，避免早期加载不必要的 Bean
2）Micrometer 指标监控成为一等公民，默认暴露 `/metrics` 端点，集成 Prometheus 更顺畅
3）弃用了部分陈旧 API，比如 `@RefreshScope` 在 Kubernetes 场景下用 ConfigMap + Reloader 更合适

代码上变化不大，但生态组件要跟上。比如你用的是 Tomcat 9，得升到 10 才支持 Jakarta Servlet；用 JPA 的话 Hibernate 要 6+。

```
// Spring Boot 3 中常用的新写法
@Bean
public RouterFunction<ServerResponse> route(UserHandler handler) {
    return route(GET("/users/{id}"), handler::getById);
}
```

迁移时建议先用 `spring-boot-migrator` 工具扫描旧项目，自动生成兼容性报告，能省不少事。

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*